

Command Line Arguments in C# with Examples

In this article, I am going to discuss **Command Line Arguments in C#** with Examples. Please read our previous article where we discussed [Input and Output in C#](#) with Examples.

Command Line Arguments in C#:

We know that we can pass parameters to a function as an argument but what about the **Main(string[] args)** method? Can we pass parameters to the Main() method in C#? Yes, we can pass parameters to the Main() method and this is possible through Command Line Arguments in C#. The arguments which are passed by the user or programmer to the Main() method are termed as Command-Line Arguments in C#.

The Main() method is the starting point from where the program execution starts. The most important point that you need to remember is that the main method doesn't accept any parameter from any method. It only accepts parameters through the Command-Line. If you notice the Main method signature, it has a string array type parameter that can accept n number of parameters at runtime. In Main(string[] args), args is a string type of array that can hold numerous parameters.

Passing Command Line Arguments in C# using Visual Studio:

Create a new console application and then modify the Program.cs class file as follows:

```
using System;
namespace FirstProgram
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"First Command Line Argument {args[0]}");
            Console.WriteLine($"Second Command Line Argument {args[1]}");
            Console.WriteLine($"Third Command Line Argument {args[2]}");
            Console.ReadLine();
        }
    }
}
```

If you notice, the above example expects at least three parameters to be supplied by the Main method. Now, if you run the application, then you will get the following System.IndexOutOfRangeException: 'Index was outside the bounds of the array' run time exception.

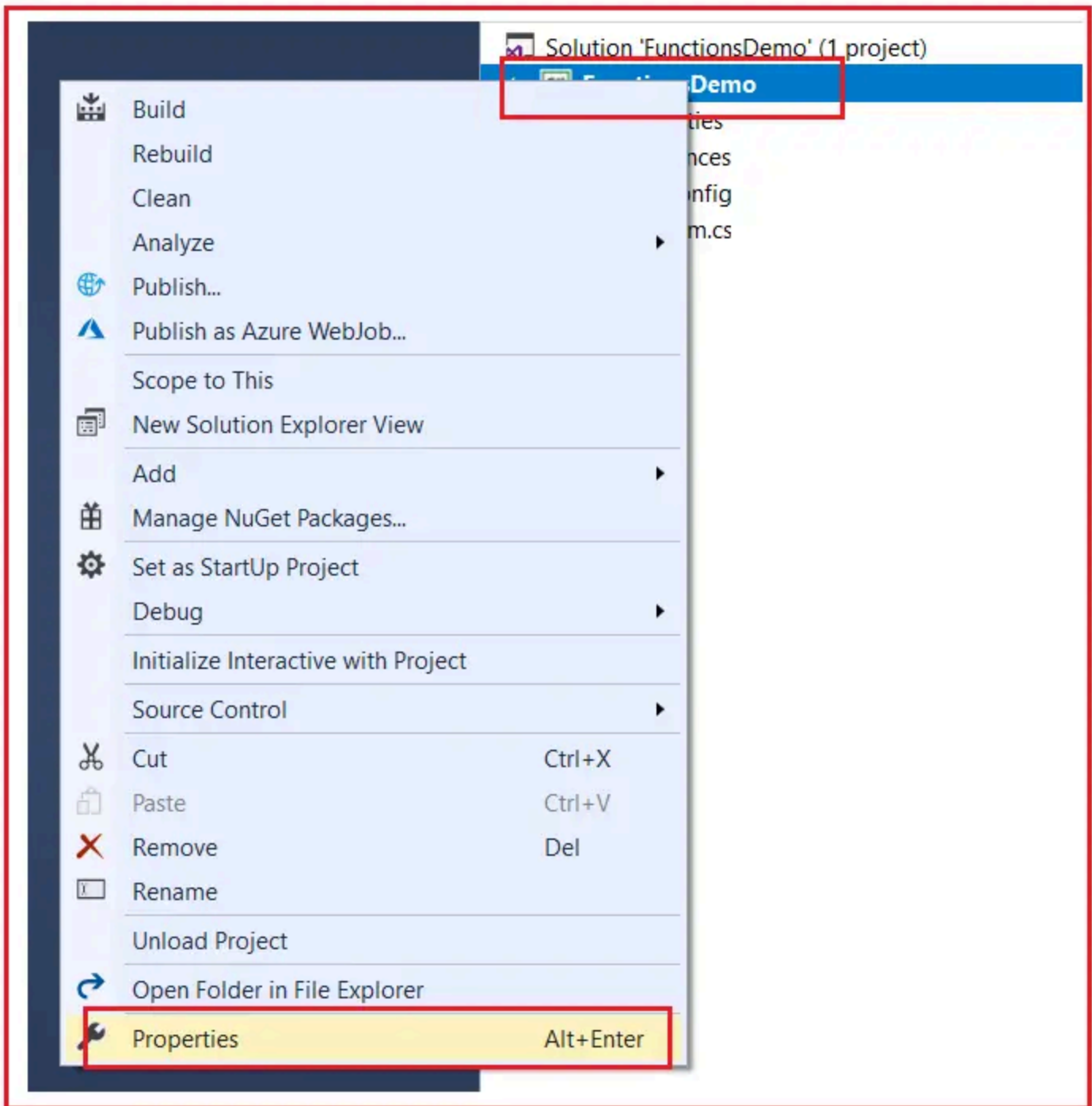
```
using System;
namespace FirstProgram
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"First Command Line Argument {args[0]}");
            Console.WriteLine($"Second Command Line Argument {args[1]}");
            Console.WriteLine($"Third Command Line Argument {args[2]}");
        }
    }
}
```



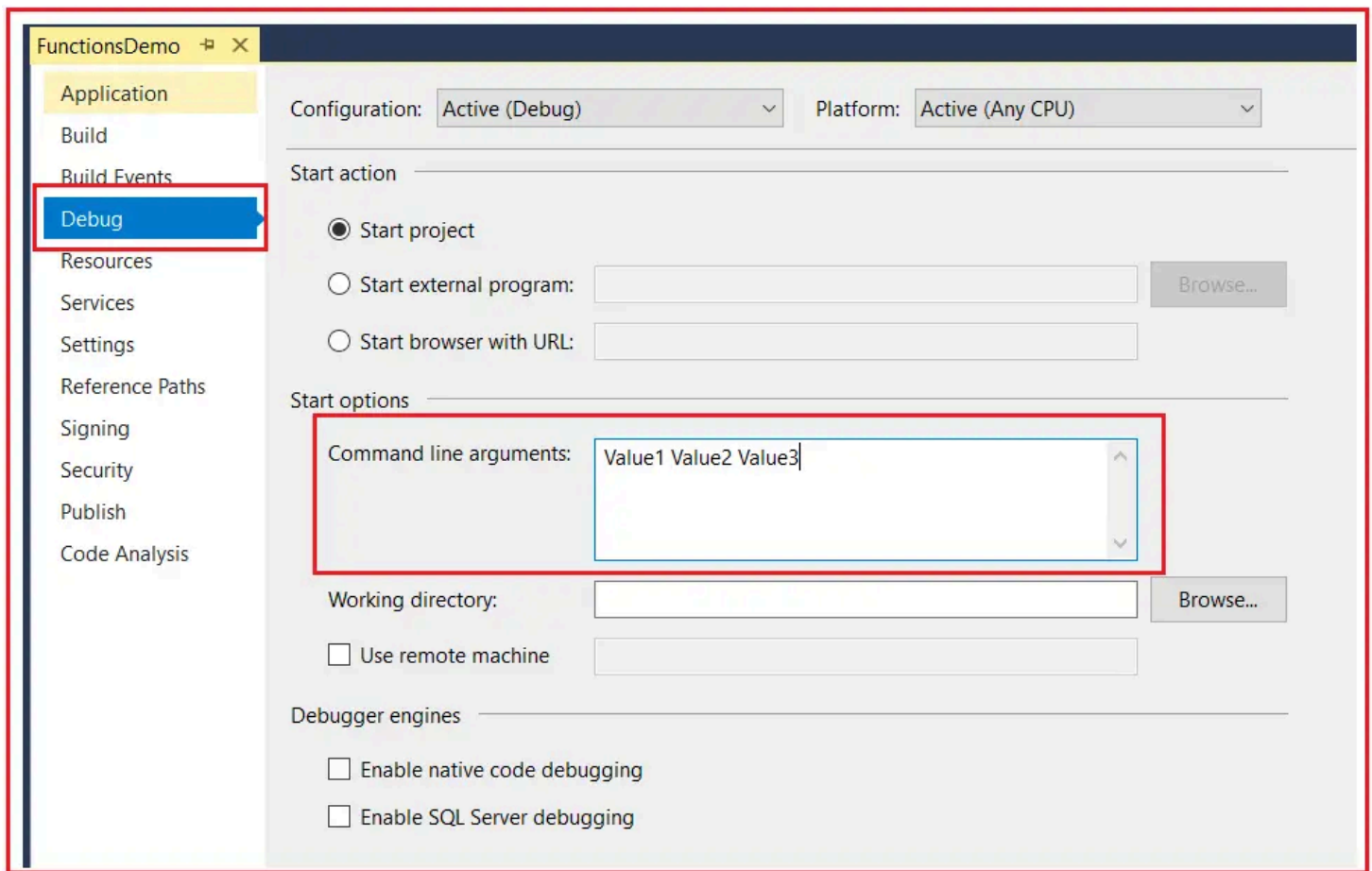
And this makes sense. Because we have not supplied any parameters and in the program, the string array does not have any element, it is empty and we are trying to access the array elements. Now, the question is how we can pass arguments to the Main Method. The answer is by using the command Line. Let us see how we can do this using Visual Studio.

Passing Command Line Arguments to Main Method using Visual Studio:

Open the Properties window. To open the properties window, right-click on the project in solution explorer and then click on the Properties menu as shown in the below image.



From the Properties window, select the debug tab and in the Command Line Arguments text box, provide the values that you want to pass to the Main method separated by a space. As in our example, we except three values in the string array, so here I am putting three values in the Command Line Arguments text box as shown in the below image.



Here Value1 will store in args[0], Value2 will store in args[1], and Value3 will store in args[2]. Now, save the changes and run the application and you will get the following output in the Console window.

```
First Command Line Argument Value1
Second Command Line Argument Value2
Third Command Line Argument Value3
```

Important Points:

1. Command Line Arguments are captured into the string array i.e. args parameter of the Main method.
2. In General, the Command Line Arguments are used to specify configuration information while launching your application.
3. Information is passed as strings.
4. There is no restriction on the number of command line arguments. You can pass 0 or n number of command line arguments.

Passing Numeric Command Line Arguments in C#

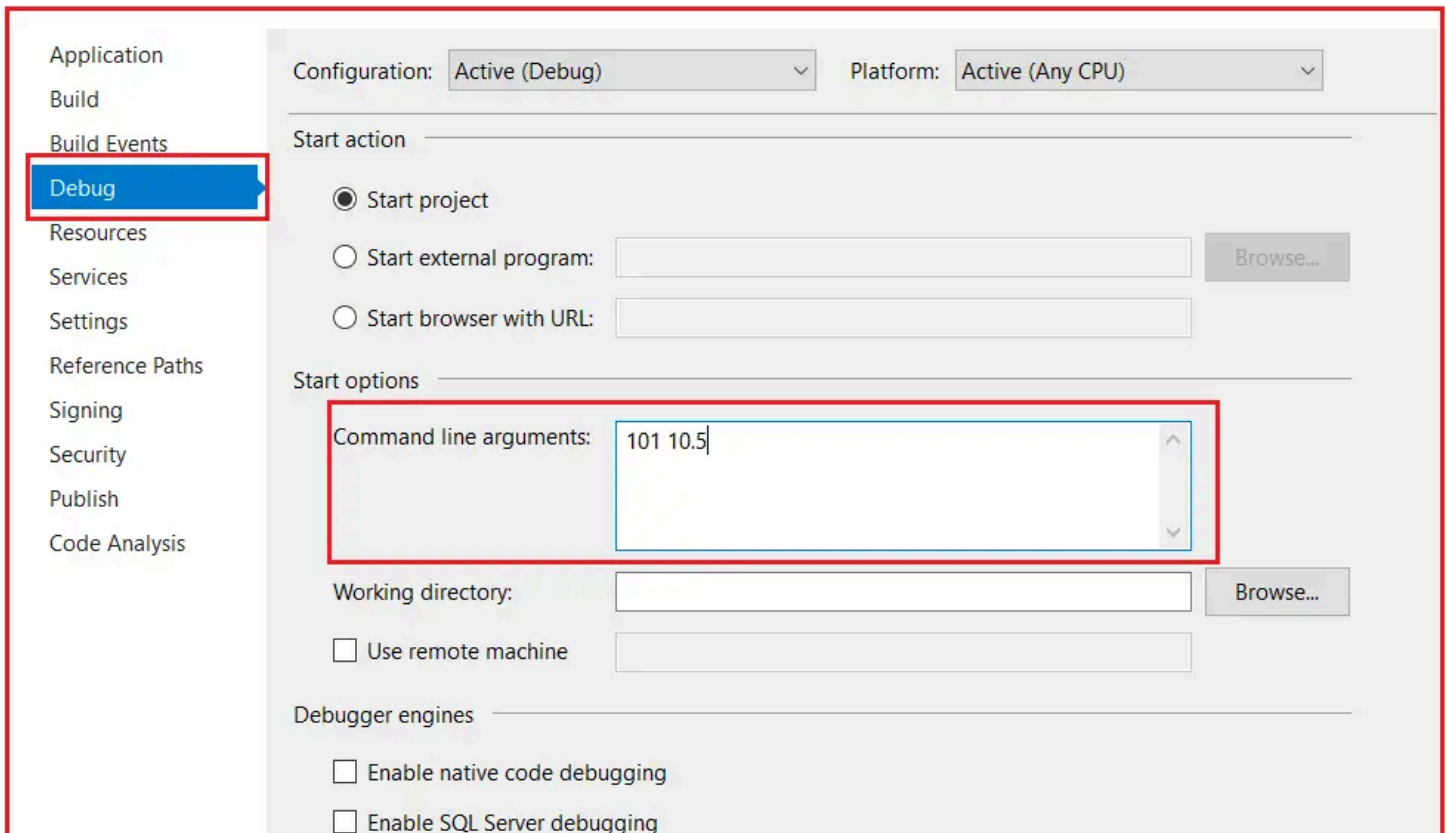
In C#, Command Line Arguments are always stored as strings and always separated by spaces. The Main() method of every C# application can only accept string arguments. If an application needs to support a numeric command-line argument, then what do you need to do? You need to pass the numeric number as a string and in your application, it is your responsibility to convert that string to numeric. And, hence it is possible to pass numeric arguments through the command line. However, we can later convert string arguments into numeric values.

Example to Pass Numeric Command Line Arguments in C#

```
using System;
namespace FirstProgram
{
    class Program
    {
        static void Main(string[] args)
        {
            //convert into integer type
            int argument1 = Convert.ToInt32(args[0]);
            Console.WriteLine("Argument in Integer Form : " + argument1);
            //convert into double type
            double argument2 = Convert.ToDouble(args[1]);
            Console.WriteLine("Argument in Double Form : " + argument2);

            Console.ReadLine();
        }
    }
}
```

Now, modify the Properties=>Debug window as shown in the below image.



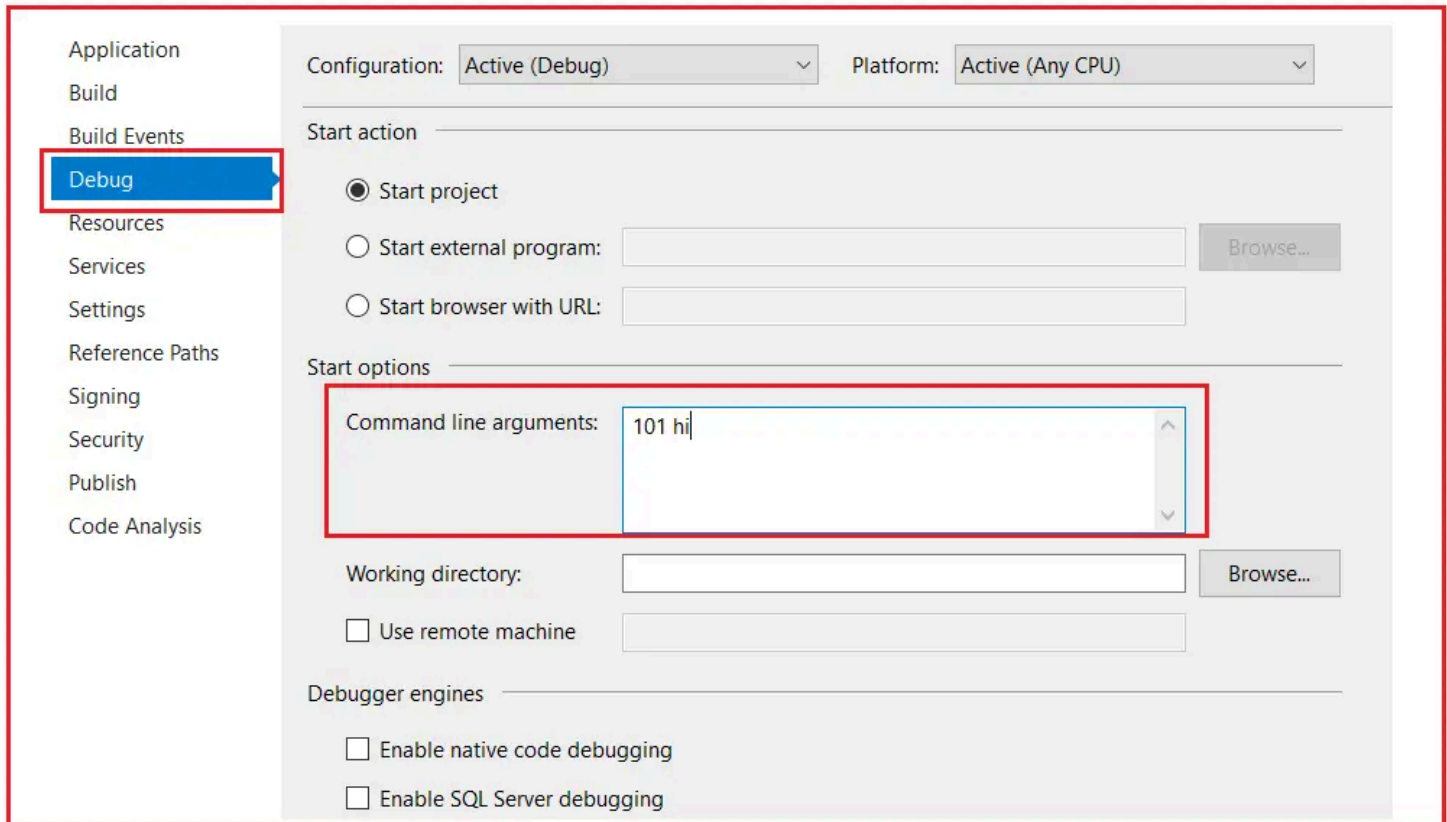
Now, save the changes and run the application and you will get the following output.

```
Argument in Integer Form : 101  
Argument in Double Form : 10.5
```

What happens if the value is not converted to the specified type?

If the arguments cannot be converted into the specified numeric value, then we will get the **System.FormatException: 'Input string was not in a correct format.'**

Let us change the command line arguments values as shown in the below image. Here the second argument is of string type which cannot be converted to double.



Now save the changes and run the application and you will get the following runtime error.

```
class Program
{
    static void Main(string[] args)
    {
        //convert into integer type
        int argument1 = Convert.ToInt32(args[0]);
        Console.WriteLine("Argument in Integer Form : " + argument1);

        //convert into double type
        double argument2 = Convert.ToDouble(args[1]);
        Console.WriteLine("Argument in Double Form : " + argument2);

        Console.ReadLine();
    }
}
```

Exception Unhandled

System.FormatException: 'Input string was not in a correct format.'

[View Details](#) | [Copy Details](#)

▸ Exception Settings

In the next article, I am going to discuss **Strings in C#** with Examples. Here, in this article, I try to explain **Command Line Arguments in C#** with Examples. I hope you enjoy this Command-Line Arguments in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.